STM32H7 中的 Cache Level1 配置相关话题

关键字：Cache， MPU

1. Cache 的不正确使用很容易造成代码执行问题

在协助客户处理问题时经常发现客户在使能指令和数据 cache 后，不注意 cache 的使用而造成很多代码的运行问题。

2. 曾经遇到过的一个案例

客户在外部使用了 3 个相同的外设，使用 SPI 模式进行通讯，调试发现最后一个初始化过的外设总是工作不正常，检查发现对最后这个外设的初始化配置没有生效，造成工作不正常。

经过调试发现客户在使能 Cache 后，没有进行任何操作，没有配置 MPU，也没有对 cache 进行冲刷。造成最后一个设备初始化的配置一直没有生效。

3. Cache 的使用概念和 MPU 配置

3.1. Cache 的使用概念

cache 总概念	微控制器的频率大幅提高了，可是一般主存储器速度无法跟上处理器的需求。如果指令和数据都存放在主存储中，主存储器的速度将会严重制约整个系统的性能。因此，高性能的微控制器会在主存储器和 CPU 之间增加高速缓冲存储器（Cache），目的是提高对存储器的平均访问速度，从而提高存储系统的性能。
时间局部性	高速缓冲存储器是全部用硬件来实现的，当 CPU 读取数据或者指令时，它同时将读取到的数据或者指令保存到一个 cache 块中。这样当 CPU 第 2 次需要读取相同的数据时，它可以从相应的 cache 块中得到相应的数据。通常，在程序中相邻的一段时间内 CPU 访问相同数据的概率是很大的，这种规律称为时间局部性。

空间局部性	当 CPU 从主存储器中读取一个字的数据时，它将会把主存储器中和 cache 块同样大小的数据读取到 cache 的一个块中。比如，如果 cache 的块大小为 4 个字，当 CPU 从主存储器中读取地址为 n 的字数据时，它同时将地址为 n、n+1、n+2、n+3 的 4 个字的数据读取到 cache 中的一个块中。这样，当 CPU 需要读取地址为 n、n+1、n+2 或者 n+3 的数据时，它可以从 cache 中得到该数据，系统的性能将得到很大的提高。在程序中，CPU 访问相邻的存储空间的数据的概率是很大的，这种规律称为空间局部性。
命中 Cache Hit	要访问的数据/指令已经存在高速缓存里【后面简称缓存】
缺失 Cache Miss	要访问的数据/指令不在高速缓存里【后面简称缓存】
I -Cache	用于指令预取的 cache 称为指令 cache，Cortex-M7 有独立 I-Cache
D-Cache	用于数据读写的 cache 称为数据 cache，Cortex-M7 有独立 D-Cache
Cache Line	Cache 与主存储器之间以块（cacheline）为单位进行数据交换，Cache 在逻辑上被划分为若干 cacheline，对应着一组存储器的位置，因此，Cache 与主存储器交换数据的最小粒度就是 cache line。
Write-Allocate 写操作分配 WA	当进行数据写操作时,如果 cache 未命中,cache 系统将会进行 cache 内容预取,从主存中将相应的块读取到 cache 中相应的位置，并执行写操作，把数据写入到 cache 中。对于写通类型(write-through)的 cache，数据将会同时被写入到主存中(写到 cache 和 memory 中)，对于写回类型(write-back)的 cache 数据将在合适的时候写回到主存中（只写到 cache 中，合适的时候才写到 memory 中）。 由于写操作分配 cache 增加了 cache 内容预取的次数，它增加了写操作的开销，但同时可能提高 cache 的命中率，因此这种技术对于系统的整体性能的影响与程序中读操作和写操作数量有关。
Write-back 回写 WB	写数据时，只更新缓存，然后将 cache line 标记为“dirty”，当这个缓存行被新的缓存行替换，或者手动 clean 的时候，再将数据写到存储器中.
Write-through	写数据时同时更新缓存（cache））和二级存储（通常是 memory），缓存行不被标记为“dirty”。这样，这对于数据一致性来说更安全，但它需要更多的总线访问。当某一个 cache line 需要替换时，就不必将其中的数据写到主存储器中去了，新调入的块可以立即把这一块覆盖掉。 Although Write-Through cache policies use more energy and are slower due to the increased bus traffic, they have two advantages over using write-back cache policies: •Write-Through cache policies reduce the need for software cache

	maintenance. •Write-Through cache policies avoid the extra decrease in determinism because there is never any dirty data in the cache.
Dirty 标记和 Eviction	Dirty 是标记那些需要写回到存储器中的缓存数据。当一个“dirty”的缓存行被新的缓存行替代时，就需要从缓存中移除一个缓存行（cache line)为新的数据腾位置，这个过程称为驱逐（Eviction）。

Cache 的读策略：

对于 cacheable 的传输，读首先要在 cache 中查找。

如果 Hit：

则 CPU 直接从 Cache 中读取数据即可。

如果 Miss，有下面两种处理方式：

Read allocate：将读回的数据在 cache 中分配一个 cache line 进行存储，再从 Cache 中读数据。占用 Cache。

Read through（也有称为 Read non-allocate）：将读回的数据直接给 master 使用，不在 cache 中缓存。避免 Cache 占用，但是对统一内容连续读效率很低。

默认情况下都是使用 read allocate（无论是 write-back 还是 write-through，他们的读取策略是一样的）。

Cache 的写策略与 write allocate / write non-allocate

Hit 时：

Write-through：将写的数据更新到 cacheline，同时将这个数据写到内存中。此模式的优点是操作简单，保证数据一致性；缺点是因为数据修改需要同时写入存储，数据写入速度较慢，所需带宽更大。

Write-back：直接将数据更新到 cacheline，不写到内存，在这个 cacheline 被驱逐出 cache 时（flush），才写入到内存中。 优点是写入速度非常快，多次到 Cache 的写只需一次到 Main Memory 的写，所需带宽更小。缺点是 Main Memory 数据和 Cache 中不一致，另外 Cache 替换的读操作会将 dirty 块写入 Main Memory。

Miss 时要考虑 allocate 配置：

Write allocate 配置时：先在 cache 中分配一个 cacheline，将数据读到 cache 中，然后依照 write-back 和 write-through 回写数据（具有 write-through 策略和 write allocate 的缓存在缓存 Miss 时从内存中读取整个块（cacheline），并仅将更新的项写入存储的内存中。驱逐不需要写入内存）。

Write non-allocate 配置：无论 write-back 还是 write-through 都直接将数据写到内存中，不会被加载到 Cache 中。只有读操作会缓存。

3.2. Cache 的使用图解

CPU 读 Cache 时：Hit 时，直接从 cache 中读取。如果 miss，则 Read through 或 Read allocate。

CPU 写时：Hit 时使用 Write Through 或 Write-back。如果 miss：使用 Write allocate 或是 No write allocate，同时配合 Write Through 或 Write-back 策略。

Write 策略组合：

Write Through (hit) + Write Allocate(miss)

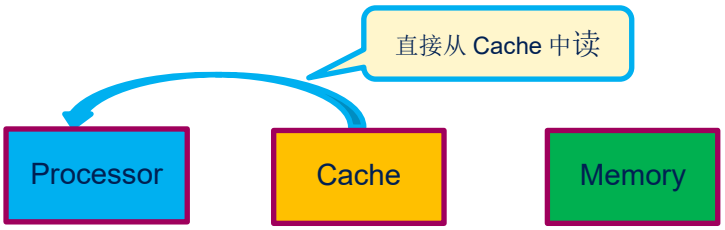
Write Through (hit) + No Write Acllocate(miss)

Write Back (hit) + Write Allocate (miss)

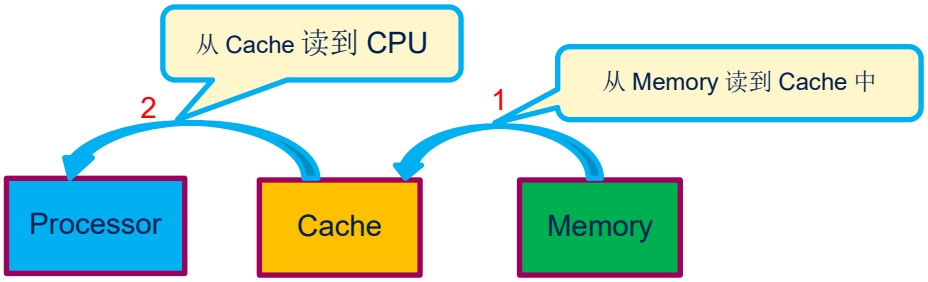
Write Back(hit) + No Write Allocate (miss)

在有 cache 的单机系统中，通常有两种写策略：write through 和 write back。这两种写策略都是针对写命中（write hit）情况而言的：write through 是既写 cache 也写 main memory；write back 是只写 cache，并使用 dirty 标志位记录 cache 的修改，直到被修改的 cache 块被替换时，才把修改的内容写回 main memory。那么在写失效（write miss）时，即所要写的地址不在 cache 中，该怎么办呢？一种办法就是把要写的内容直接写回 main memory，这种办法叫做 no write allocate policy；另一种办法就是把要写的地址所在的块先从 main memory 调入 cache 中，然后写 cache，这种办法叫做 write allocate policy。

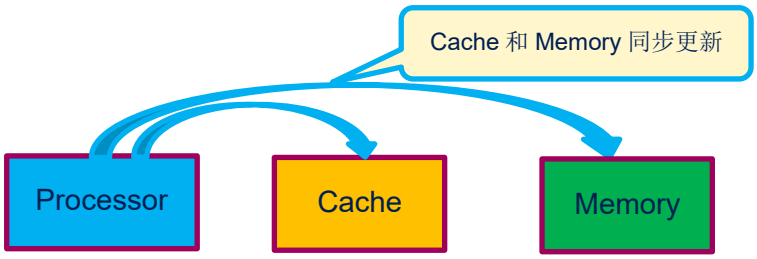
- 读命中(Hit):



- Read Allocate（读分配, 先把数据读到 cache 中，再从 cache 中读到 CPU。cache 使能时，默认都是使用 read allocate）：



- Write Through（WT，写通,透写，直接写，Hit 时把数据同时写到 cache 和内存中）：



写数据时同时更新缓存和二级存储，缓存行不被标记为“dirty。这样，这对于数据一致性来说更安全，但它需要更多的总线访问。当某一个 cache line 需要替换时，就不必将其中的数据写到主存储器中去了，新调入的块可以立即把这一块覆盖掉。

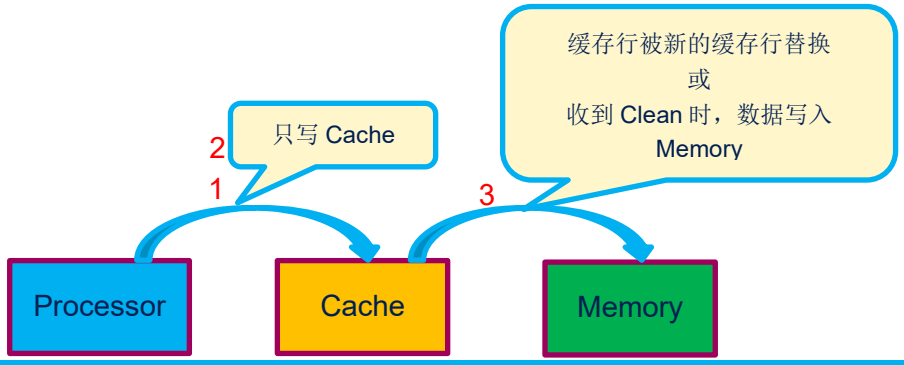
尽管由于总线流量的增加，Write-Through 缓存策略使用更多的能量并且速度更慢，但与使用回写缓存策略相比，它们有两个优点：

•Write-Through cache policies reduce the need for software cache maintenance.

•Write-Through cache policies avoid the extra decrease in determinism because there is never any dirty data in the cache.

- Write Back（hit 时）：

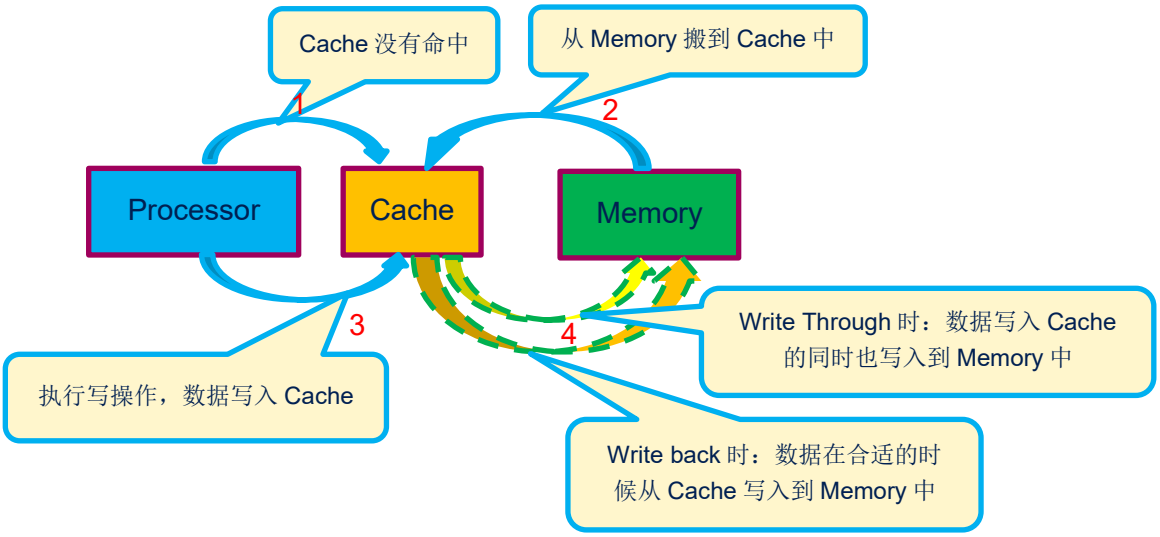
写数据时，只更新缓存，然后将 cache line 标记为“dirty”，当这个缓存行被新的缓存行替换，或者手动 clean 的时候，再将数据写到存储器中.



- Write Allocate（WA，写分配，Miss 时考虑）：

当进行数据写操作时,如果 cache 未命中,cache 系统将会进行 cache 内容预取,从主存中将相应的块读取到 cache 中相应的位置，并执行写操作，把数据写入到 cache 中。对于写通类型的 cache，数据将会同时被写入到主存中，对于写回类型的 cache 数据将在合适的时候写回到主存中(写在此时只发生在 cache 中)。

由于写操作分配 cache 增加了 cache 内容预取的次数，它增加了写操作的开销，但同时可能提高 cache 的命中率，因此这种技术对于系统的整体性能的影响与程序中读操作和写操作数量有关。



Write non-Allocate（Miss 时考虑）：

Cache miss 时，无论是 write through 还是 write back，都是直接将数据写入内存中。不加载到 cache。

3.3. Cache 控制寄存器

Table A3-4 Memory attribute summary

Memory type attribute	Shareability	Other attributes	Description
Strongly-ordered	Shareable	-	All memory accesses to Strongly-ordered memory occur in program order. All Strongly-ordered regions are Shareable.
Device	Shareable	-	Intended to handle memory- mapped peripherals that are shared by several processors.
	Non-shareable	-	Intended to handle memory- mapped peripherals that are used only by a single processor.
Normal	Shareable	Cacheability, one of: [a] • Non-cacheable Write-Through cacheable. • Write-Back Write-Allocate cacheable. • Write-Back no Write-Allocate cacheable.	Intended to handle Normal memory that is shared between several processors.
	Non-shareable		Intended to handle Normal memory that is used by only a single processor.

a. The cacheability attribute is defined independently for inner and outer cache regions.

4.9.3 L1 Cache Control Register

Note

This register is optional and might not be available in your implementation.

The CACR controls the optional L1 ECC and the L1 cache coherency usage model. The bit assignments are:

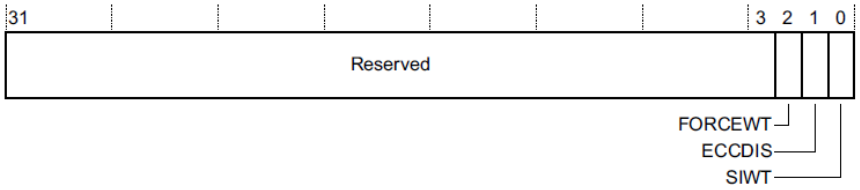


Table 4-71 CACR bit assignments

Bits	Name	Type	Function
[31:3]	-	-	Reserved, RAZ/WI.
[2]	FORCEWT	RW	Enables Force Write-Through in the data cache: **0** Disables Force Write-Through. **1** Enables Force Write-Through. All Cacheable memory regions are treated as Write-Through. This bit is RAZ/WI if the data cache is excluded. If the data cache is included the reset value of FORCEWT is 0.
[1]	ECCDIS	RW	Enables ECC in the instruction and data cache: **0** Enables ECC in the instruction and data cache. This is RAO/WI if both data cache and instruction cache are excluded or if ECC is excluded. **1** Disables ECC in the instruction and data cache. This is RAZ/WI if data cache is not configured.
[0]	SIWT	RW	Shared cacheable-is-WT for data cache. Enables cache coherency usage: **0** Normal Cacheable Shareable locations are treated as being Non-cacheable. Programmed inner cacheability attributes are ignored. This is the default mode of operation for shared memory. The data cache is transparent to software for these locations and therefore no software maintenance is required to maintain coherency. **1** Normal Cacheable Shareable locations are treated as Write-Through. Programmed inner cacheability attributes are ignored. All writes are globally visible. Other memory agent updates are not visible to Cortex-M7 processor software without suitable cache maintenance. Useful for heterogeneous MP-systems where, for example, the Cortex-M7 processor is integrated on the *Accelerator Coherency Port* (ACP) interface on an MP-capable processor. This bit is RAZ/WI when data cache is not configured.

SIWT=0: 所有的 shared 位置被认为是 Non-cacheable 的(例如 Dcache)，配置的内部可缓存性属性将被忽略掉。（share 了的就不能 cache，这是共享内存的默认操作模式。cache 对这些位置的软件是透明的，因此不需要软件维护来保持一致性。）

SIWT=1:普通的、可缓存的、共享的位置被视为 WT（write-through），配置的其他属性不受 CACR.SIWT 的影响，仍然是 WT 特性。对于 I-cache：共享（shared）位置被视为不可缓存（non-cacheable）。

3.4. MPU 的使用概念

在 STM32H7 的 PM0253 编程手册中 MPU 配置:

4.6.6 MPU access permission attributes

This section describes the MPU access permission attributes. The access permission bits, TEX, C, B, S, AP, and XN, of the RASR, control access to the corresponding memory region. If an access is made to an area of memory without the required permissions, then the MPU generates a permission fault. *Table 91* shows encodings for the TEX, C, B, and S access permission bits.

Table 91. TEX, C, B, and S encoding

TEX	C	B	S	Memory type	Shareability	Other attributes
0b000	0	0	x(1)	Strongly-ordered	Shareable	-
		1	x(1)	Device	Shareable	-
	1	0	0	Normal	Not shareable	Outer and inner write-through. No write allocate.
			1		Shareable	
		1	0	Normal	Not shareable	Outer and inner write-back. No write allocate.
			1		Shareable	
0b001	0	0	0	Normal	Not shareable	Outer and inner noncacheable.
			1		Shareable	
		1	x(1)	Reserved encoding		-
	1	0	x(1)	Implementation defined attributes.		-
		1	0	Normal	Not shareable	Outer and inner write-back. Write and read allocate.
			1		Shareable	
0b010	0	0	x(1)	Device	Not shareable	Nonshared Device.
		1	x(1)	Reserved encoding		-
	1	x(1)	x(1)	Reserved encoding		-

Table 91. TEX, C, B, and S encoding (continued)

TEX	C	B	S	Memory type	Shareability	Other attributes
0b1BB	A	A	0	Normal	Not shareable	Cached memory, BB = outer policy, AA = inner policy. See *Table 92 on page 229* for the encoding of the AA and BB bits.
			1		Shareable	

上表中关于 Cache 策略的设置 AA/BB 定义如下:

00 Non-cacheable

01 Write-back, write and read allocate

Bits	Name	Description
28	XN	Execute never
26:24	AP	Data access permission field (RO, RW, or No access)
21:19	TEX	Type extension field
18	S	Shareable
17	C	Cacheable
16	B	Bufferable
15:8	SRD	Subregion disabled. For each subregion 1 = disabled, 0 = enabled.
5:1	SIZE	Specifies the size of the MPU protection region.

10 Write-through, no write allocate

11 Write-back, no write allocate

概念：TEX，C,B, S, memory type,shareability, attributes. Write Through，Write Back，Write Allocate，No Write Allocate, Read Allocate

内存类型：Normal Memory, Device Memory, Strongly Ordered Memory。Device 和 Strongly-ordered 类型的存储器（可以理解为外设）区域，都是 Non-cacheable 的。见上表 “Other attributes” 栏。

缓存策略 cacheability：Non-Cacheable（主控不使用 Cache，直接与存储器交互，这样不会带来内存一致性问题，但是效率不高） 和 Cacheable （Cacheabe 又分为 Write-Through cacheable 透写 和 Write-Back cacheable 回写）

共享属性 shareability：共享 Shareable（比如多核 CPU 中，UART 外设地址空间是对所有 CPU 共享的），不共享 Non- shareable（当被配置成为 Non-Shareable 的时候，意味着在多核系统中，它只能被一个核心访问）

Normal Memory: 通常，用于程序代码和数据存储的内存是普通内存。使用普通内存技术的例子有:

Programmed Flash ROM /*在编程过程中，闪存可以比普通存储器更严格地排序*/ROM、 SRAM、DRAM 和 DDR memory

Device Memory: Peripherals and I/O,通常是系统外设（I/O），Device 内存类型属性定义了内存位置，在这些位置上，对这些位置的访问可能会导致副作用，或者为一个负载返回的值可能会根据执行的负载数量而变化。内存映射外设和 I/O 位置是通常标记为设备的内存区域的示例： memory-mapped 外设（比如 I/O，访问会造成一些影响。），内存控制配置寄存器，中断控制寄存器，FIFO。

Strongly Ordered Memory: 比 Normal Memory 的访问规则更严格；Peripherals and I/O,通常是系统外设（I/O），对于强顺序内存，所有内存访问都严格按照内存访问指令的程序顺序进行排序：读和写都会对其他方面造成一定影响；访问不能重复；必须维护访问的数量、顺序和大小。具有 Shareable 特性。

3.5. ST 的参考文件

AN4839：Level 1 cache on STM32F7 Series and STM32H7 Series

AN4838：Introduction to memory protection unit management on STM32 MCUs

下图是 MPU 没使能，并且没有对某些特殊寄存器进行配置，那么存储器的映射地址及其属性就可以参考下图。

其中，WT 表示 Write-through（透写），WB 表示 Write-back（回写），WA 表示 Write-allocate（写分配），没有明确标注 WA 的就是 RA（读分配）。

图1. AN4839 中 STM32F7 和 STM32H7 default settings

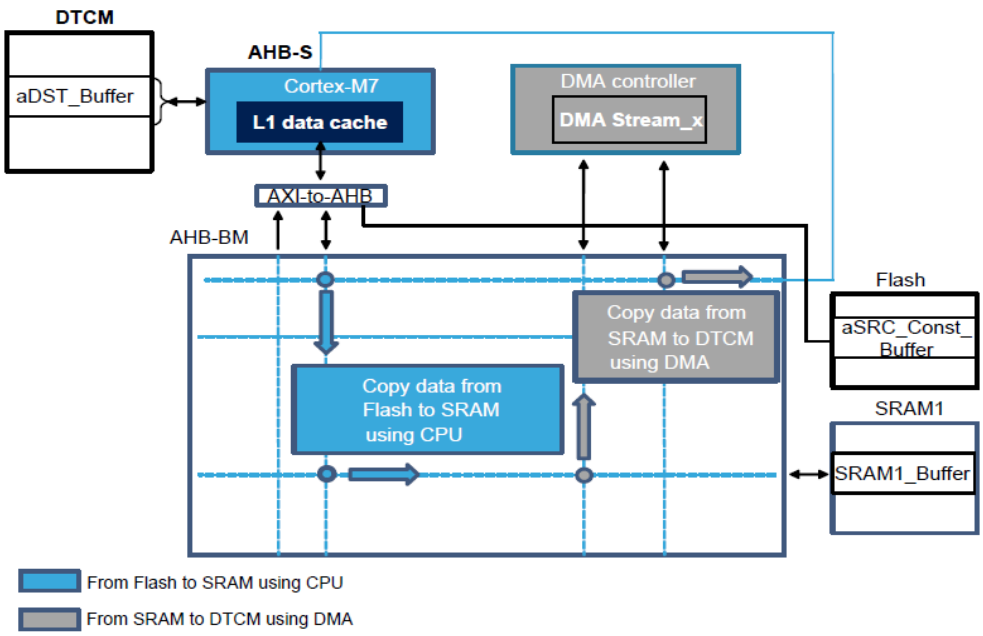
By default, the MPU is disabled. In this case, the cache setting is defined as a default address map.

Table 2. **Memory region shareability and cache policies**

Address range	Memory region	Memory type	Shareability	Cache policy
0x00000000-0x1FFFFFFF	Code	Normal	Non-shareable	WT
0x20000000-0x3FFFFFFF	SRAM	Normal	Non-shareable	WBWA
0x40000000-0x5FFFFFFF	Peripheral	Device	Non-shareable	-
0x60000000-0x7FFFFFFF	External RAM	Normal	Non-shareable	WBWA
0x80000000-0x9FFFFFFF				WT
0xA0000000-0xBFFFFFFF	External Device	Device	Shareable	-
0xC0000000-0xDFFFFFFF			Non-shareable	
0xE0000000-0xE00FFFFF	Private peripheral bus	Strongly-ordered	Non-shareable	-
0xE0100000-0xFFFFFFFF	Vendor-specific system	Device	Non-shareable	-

请参考 AN4839 文件末尾的 cache 和 MPU 使用示例（下图）：PC 将 Flash 中的数据拷贝到 RAM，DMA 将 SRAM 中的数据拷贝到 DTCM 中，最后比较 DTCM 和 Flash 中的原始数据来理解 Cache、MPU 和数据一致性。

Figure 2. **Data transfer paths**



数据传输中如果配置为 write-back，即写数据时，只更新缓存，然后将 cache line 标记为“dirty”，当这个缓存行被新的缓存行替换，或者手动 clean 的时候，再将数据写到存储器中，这样容易造成数据的不一致。

要避免这种问题，需要：

- **Solution 1:** to perform a cache maintenance operation after writing data to a cacheable memory region, by forcing a D-cache clean operation by software through CMSIS function SCB_CleanDCache() (all the dirty lines are write-back to SRAM1).
- **Solution 2:** in order to ensure the cache coherency, the user must modify the MPU attribute of the SRAM1 from write-back (default) to write-through policy.
- **Solution 3:** to modify the MPU attribute of the SRAM1 by using a shared attribute. This prevents by default the SRAM1 from being cached in D-cache.
- **Solution 4:** to perform a cache maintenance operation, by forcing write-through policy for all the writes. This can be enabled by setting force write-through in the D-Cache bit in the CACR control register.

The data coherency between the core and the DMA is ensured by:

1. Either making the SRAM1 buffers not cacheable
2. Or making the SRAM1 buffers cache enabled with write-back policy, with the coherency ensured by software (clean or invalidate D-Cache)
3. Or modifying the SRAM1 region in the MPU attribute to a shared region.
4. Or making the SRAM1 buffer cache enabled with write-through policy.

Note：MPU 配置为 shareable，相当于没使用 cache 的效果。

如果系统对执行速度要求比较高，建议使用 cache，然后通过 cache 操作函数维护数据的一致性。

AN4838: MPU 寄存器的使用信息，可以参考 AN4838 文件中提到的 Programmer manual：

For more details about the MPU, refer to the following documents available on www.st.com

- Programming manual *STM32F7 series and STM32H7 series Cortex®-M7 processor* (PM0253)
- Programming manual *STM32F10xxx/20xxx/21xxx/L1xxxx Cortex®-M3* (PM0056)
- Programming manual *STM32 Cortex®-M0+ MCUs programming manual* (PM0223)
- Programming manual *STM32 Cortex®-M4 MCUs and MPUs* (PM0214)
- Programming manual *STM32 Cortex®-M33 MCUs* (PM0264)

具体配置和示例代码

在 AN4838 中有如下示例代码：

图2. Example of setting up the MPU

Configuration	Memory type	Base address	Region number	Memory size	Memory attributes
Internal SRAM	Normal memory	0x2000 0000	Region0	8 Kbytes	Shareable, write through, no write allocate C = 1, B = 0, TEX = 0, S = 1 SRD = 0, XN = 0, AP = full access
Flash memory	Normal memory	0x0800 0000	Region1	1 Mbyte	Non-shareable write through, no write allocate C = 1, B = 0, TEX = 0, S = 1 SRD = 0, XN = 0, AP = full access
FMC	Device memory	0x4000 0000	Region2	512 Mbytes	Shareable, write through, no write allocate C = 1, B = 0, TEX = 0, S = 1 SRD = 0, XN = 1, AP = full access

图3. Setting the MPU with STM32Cube HAL

```
void MPU_RegionConfig(void)
{
MPU_Region_InitTypeDef MPU_InitStruct;
/* Disable MPU */
HAL_MPU_Disable();
/* Configure RAM region as Region N°0, 8kB of size and R/W region */
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0x20000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_8KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER0;
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);
/* Configure FLASH region as REGION N°1, 1MB of size and R/W region */
MPU_InitStruct.BaseAddress = 0x08000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_1MB;
MPU_InitStruct.Number = MPU_REGION_NUMBER1;
HAL_MPU_ConfigRegion(&MPU_InitStruct);
/* Configure FMC region as REGION N°2, 0.5GB of size, R/W region */
MPU_InitStruct.BaseAddress = 0x40000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_512MB;
MPU_InitStruct.IsShareable = MPU_ACCESS_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER2;
```

```
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_DISABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);
/* Enable MPU */
HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);
}
```

最高性能配置：MPU enable, MPU_TEX_Level1, bufferable, cacheable, not shareable

3.6. MPU 配置和 cache 相关函数

使能 cache 的函数：

SCB_EnableDCache();

SCB_EnableICache();

使能 cache 后，要配合 MPU 的正确配置来使系统性能最佳。

3.6.1. MPU 配置示例：

内部 Flash：仅运行指令，最低性能的配置，读写都不使用 cache：

MPU_ACCESS_NOT_BUFFERABLE，

MPU_ACCESS_NOT_CACHEABLE，

MPU_ACCESS_SHAREABLE,

MPU_TEX_LEVEL1. TEX=0b001, C=0, B=0, S=0 or 1.

最高性能的配置：

MPU_ACCESS_BUFFERABLE，

MPU_ACCESS_CACHEABLE，

MPU_ACCESS_NOT_SHAREABLE，

MPU_TEX_LEVEL1。TEX=0b001, C=1, B=1, S=0, shareability =Not shareable, -> outer and inner write-back, write and read allocate.

Flash 高性能配置*/

```
MPU_InitStruct.Enable = MPU_REGION_ENABLE;  //使能该保护区域
MPU_InitStruct.BaseAddress = 0x08000000;   //设置基地址
MPU_InitStruct.Size = MPU_REGION_SIZE_2M;  //保护区域的大小
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS; //设置访问权限
MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;     // 允许缓冲 buffer
MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;   //允许 cache
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;    // 不共用 not share
MPU_InitStruct.Number = MPU_REGION_NUMBER0;   //设置包含区域
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL1;  //设置类型扩展域
MPU_InitStruct.SubRegionDisable = 0x00;  //禁止子区域
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE; //允许读取指令
HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

/*DTCM 和 ITCM，和 CPU 主频一样，不需要配置 MPU 和 cache。

AXI SRAM: 配置为 Normal， cache 读写都开启；性能最强，缺点：由于开启了读 cache 和写 cache，需要用户调用函数 SCB_CleanInvalidataDCache(),管理数据一致性，需要注意，只有多主控操作（包括 DMA）此空间需要如此处理。*/

```
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0x24000000;
```

```
MPU_InitStruct.Size = MPU_REGION_SIZE_512KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER1;
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);

/*对于 Device 或 Strongly order 的外设，读写 cache 都要关闭，比如寄存器，FMC 扩展IO，FMC驱动的一些strongly order外设，LCD，并口NOR Flash，并口SRAM，NAND Flash 等。
能保证严格按照程序代码执行，缺点是不支持非对齐访问。·
示例代码如下：*/
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0x60000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_64KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_BUFFERABLE;
MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER2;
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);

/*配置为 Normal，关闭写 Cache，开启读 cache，
特点：保证数据直接写到 SDRAM，保证读性能最佳
缺点：由于开启了读 cache，需要用户调用函数 SCB_CleanInvalidataDCache(),管理数据一致性问题，这里需要注意，只有多主控操作此空间需要处理（包括 DMA）。*/
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0xC0000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_32MB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
```

```
MPU_InitStruct.IsCacheable = MPU_ACCESS_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER3;
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL0;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);

/*配置为最低性能 Normal，关闭 cache 的读写，normal 里性能最差*/
MPU_InitStruct.Enable = MPU_REGION_ENABLE;
MPU_InitStruct.BaseAddress = 0x24000000;
MPU_InitStruct.Size = MPU_REGION_SIZE_512KB;
MPU_InitStruct.AccessPermission = MPU_REGION_FULL_ACCESS;
MPU_InitStruct.IsBufferable = MPU_ACCESS_NOT_BUFFERABLE;
MPU_InitStruct.IsCacheable = MPU_ACCESS_NOT_CACHEABLE;
MPU_InitStruct.IsShareable = MPU_ACCESS_NOT_SHAREABLE;
MPU_InitStruct.Number = MPU_REGION_NUMBER5;    //根据具体配置 Number 号
MPU_InitStruct.TypeExtField = MPU_TEX_LEVEL1;
MPU_InitStruct.SubRegionDisable = 0x00;
MPU_InitStruct.DisableExec = MPU_INSTRUCTION_ACCESS_ENABLE;
HAL_MPU_ConfigRegion(&MPU_InitStruct);
```

MPU 的详细内容，请参考 ARM 使用手册 DUI0646C_cortex_m7_dgug（Arm Cortex-M7 Devices Generic User Guide r1p2，下面是 Cortex-M7 MPU 的寄存器：

Cortex-M7 Peripherals

Use the MPU registers to define the MPU regions and their attributes. The MPU registers are:

Table 4-48 MPU registers summary

Address	Name	Type	Required privilege	Reset value	Description
0xE000ED90	MPU_TYPE	RO	Privileged	0x00000800	*MPU Type Register*
0xE000ED94	MPU_CTRL	RW	Privileged	0x00000000	*MPU Control Register* on page 4-45
0xE000ED98	MPU_RNR	RW	Privileged	Unknown	*MPU Region Number Register* on page 4-47
0xE000ED9C	MPU_RBAR	RW	Privileged	Unknown	*MPU Region Base Address Register* on page 4-47
0xE000EDA0	MPU_RASR	RW	Privileged	-[a]	*MPU Region Attribute and Size Register* on page 4-48
0xE000EDA4	MPU_RBAR_A1	RW	Privileged	Unknown	Alias of RBAR, see *MPU Region Base Address Register* on page 4-47
0xE000EDA8	MPU_RASR_A1	RW	Privileged	-[a]	Alias of RASR, see *MPU Region Attribute and Size Register* on page 4-48
0xE000EDAC	MPU_RBAR_A2	RW	Privileged	Unknown	Alias of RBAR, see *MPU Region Base Address Register* on page 4-47
0xE000EDB0	MPU_RASR_A2	RW	Privileged	-[a]	Alias of RASR, see *MPU Region Attribute and Size Register* on page 4-48
0xE000EDB4	MPU_RBAR_A3	RW	Privileged	Unknown	Alias of RBAR, see *MPU Region Base Address Register* on page 4-47
0xE000EDB8	MPU_RASR_A3	RW	Privileged	-[a]	Alias of RASR, see *MPU Region Attribute and Size Register* on page 4-48

a. Unknown apart from the ENABLE field, which is reset to 0.

3.6.2. Cache 相关函数的使用说明

CMSIS 软件包的 core_cm7.h 文件为 Cache 的配置提供了 11 个函数：

SCB_EnableICache
SCB_DisableICache
SCB_InvalidateICache
SCB_EnableDCache
SCB_DisableDCache
SCB_InvalidateDCache
SCB_CleanDCache
SCB_CleanInvalidateDCache
SCB_InvalidateDCache_by_Addr
SCB_CleanDCache_by_Addr
SCB_CleanInvalidateDCache_by_Addr

SCB_EnableICache(void) 使能 I cache

SCB_DisableICache(void) 失能 I cache

SCB_InvalidateICache(void) 使 I cache line 标记为无效，等同于删除操作。当读取指令时，会理解成 cache-line 中没有对应指令，转而去真实的物理地址获取指令。

SCB_EnableDCache(void) 使能 D cache

SCB_DisableDCache(void) 失能 D cache

SCB_InvalidateDCache(void) 使 D cache 标记为无效，等同于删除操作，当读取数据时，会理解成 cache-line 中没有对应数据，转而去真实物理地址获取数据。

SCB_CleanDCache(void) Clean 所有的 cache-line，即将标记为 dirty 的 cache-line 数据全部写到 cache line 对应的真实的物理地址中。（比如 write-back，cache-line 中有新的数据，并且标记为 dirty，但真实物理地址中可能是旧的数据。这个 clean，会把数据回写到真实 memory 中。）

SCB_CleanInvlaidateDCache(void) 是前面两个函数 SCB_InvalidateDCache 和 SCB_CleanDCache 的二合一。将 Cache Line 中标记为 dirty 的数据写入到相应的存储区后，再将 Cache Line 标记为无效，表示删除。Cache 空间就都腾出来，可以加载新的数据。

SCB_InvalidateDCache_by_Addr() 根据地址信息无效其对应的 cache-line。

SCB_CleanDCache_by_Addr() 根据地址信息 clean 其对应的 cache-line。

SCB_CleanInvalidateDCache_by_Addr() 根据地址信息 clean 并 invalidate 其对应的 cache-line。

Cortex-M7 内核的 L1 Cache 由多行内存区组成，每行有 32 字节， 请注意 cache addr 操作的地址一定要 32 字节对齐的。dsize 一定要是 32 字节的整数倍。（Cortex m7 cache line 是 8 words，32bytes。对应的如果其他系统是 64bytes cache line 对应的就是 64bytes）

3.6.3. 使用 DMA 时软件维护一致性

启用 Cache 通常可以带来程序运行速度的提升，不过，当在代码中使用到 DMA 时，我们可能需要根据 DMA 的不同传输方向，采取相应 Cache 维护策略措施以保持多主访问时的数据一致性。

在启用 cache 并遵循 32 字节对齐的条件下：

如果 DMA 负责从 I/O 读取数据到内存(DMA Buffer)中，那么在 DMA 传输之前，可以 invalid DMA Buffer 地址范围的高速缓存。在 DMA 传输完成后，程序读取数据不会由于 cache hit 导致读取过时的数据。

如果 DMA 负责把内存(DMA Buffer)数据发送到 I/O 设备，那么在 DMA 传输之前，可以 clean DMA Buffer 地址范围的高速缓存，clean 的作用是写回 cache 中修改的数据（数据准备阶段用到的 cache line）。在 DMA 传输时，就不会把主存中的过时数据发送到 I/O 设备。

请注意，在 DMA 传输没有完成期间 CPU 不要访问 DMA Buffer（如果需要访问 DMA buffer，请注意保证数据的一致性）。

在速度要求不太严格的时候，保险起见（比如数据没有对齐，或是客户需要使用简单方法保证数据一致性），建议 DMA 传输前，对 DMA 要访问的对应地址的 cache 空间进行先 clean 后 invalid 操作（先 clean 不会损坏其他有用数据，后 invalid 操作可以理解为删除操作）。在 DMA 传输完成，并搬移 DMA buffer 内容到其他空间后，对对应地址的 cache 空间进行先 clean 后 invalid 操作。当然，本建议可根据具体应用灵活处理。

cache addr 操作的地址一定要 32 字节对齐的。dsize 一定要是 32 字节的整数倍。这点同样适用于可以 Cacheable 的 DMA buffer（dsize 为 32byts 整数倍，当 buffer 大小不是 32bytes 的整数倍的情况下有一定的空间浪费）。32 bytes 是根据 cache line 的最小粒度来确定的（Cortex m7 cache line 是 8 words，32bytes。对应的如果其他系统是 64bytes cache line 对应的就是 64bytes）。如果 DMA Buffer 的安排也遵守 32 字节对齐，就可以避免同一 Cacheline 出现两种应用场景的数据,否则不方便针对 Cacheline 做数据维护操作。

Cache 的使用能否提高效率需要综合考虑，因为维护数据一致性有一定开销：对于 32bytes，16bytes，8bytes，1bytes 的一些常用变量，不使用 cache 但把他们放在 0 延时等待的 DTCM 空间效率会更好。对与 512bytes，1024bytes 等，空间需求较大，DTCM 放置栈、堆及常用变量后可能空间比较紧张了，这些需求量大的 buffer 可以放在 AXI SRAM，SRAM1，SRAM2 等空间，使用 cache 效率会更高，但需要地址和 size 大小对齐。配置外设、同外设进行通讯、DMA 操作时也需要维护一致性或是配合 MPU 来使用。

4. 小结

在使用 cache 时要配合使用 MPU，并且做好数据一致性的处理。

参考文献

文件编号	文件标题	版本号	发布日期
Copyright © 2015, 2018 Arm. ARM DUI 0646C (ID121118)	DUI0646C_cortex_m7_dgug（Arm Cortex-M7 Devices Generic User Guide r1p2）.pdf	Copyright © 2015, 2018 Arm	
	https://community.st.com/t5/stm32-mcus-products/scb-invalidatedcache-by-addr-not-operating-correctly/td-p/118256		
	https://community.st.com/t5/stm32-mcus-products/maintaining-cpu-data-cache-coherence-for-dma-buffers/m-p/95746		

版本历史

日期	版本	变更
2025 年 09 月 29 日	1.0	首版发布

重要通知 - 请仔细阅读

意法半导体公司及其子公司 （“ST”）保留随时对 ST 产品和 / 或本文档进行变更的权利，恕不另行通知。买方在订货之前应获取关于 ST 产品的最新信息。 ST 产品的销售依照订单确认时的相关 ST 销售条款。

买方自行负责对 ST 产品的选择和使用， ST 概不承担与应用协助或买方产品设计相关的任何责任。

ST 不对任何知识产权进行任何明示或默示的授权或许可。

转售的 ST 产品如有不同于此处提供的信息的规定，将导致 ST 针对该产品授予的任何保证失效。

ST 和 ST 徽标是 ST 的商标。若需 ST 商标的更多信息，请参考 *www.st.com/trademarks*。所有其他产品或服务名称均为其 各自所有者的财产。

本文档是 ST 中国本地团队的技术性文章，旨在交流与分享，并期望借此给予客户产品应用上足够的帮助或提醒。若文中内容存有局限或与 ST 官网资料不一致，请以实际应用验证结果和 ST 官网最新发布的内容为准。您拥有完全自主权是否采纳本文档（包括代码，电路图等）信息，我们也不承担因使用或采纳本文档内容而导致的任何风险。

本文档中的信息取代本文档所有早期版本中提供的信息。

© 2020 STMicroelectronics - 保留所有权利